

AccessPresent AI

Gesture-Controlled, Hands-Free Presentation System

A browser-based computer-vision system that lets a presenter navigate and point at their slides using natural hand gestures captured from an ordinary webcam — no clicker, no installed software, no video ever leaving the device. It also adds several things most gesture-controlled presentation tools don't: live captions of the presenter's speech, an instant offline “I'm stuck” cue, a searchable slide browser for jumping straight to any slide, and an on-screen pace tracker.

Team: [team / group name] **Programme:** [degree programme] **Course:** [course / unit code] **Date:** 14 August 2026

01

Group Members & Degree Programmes

Fill in your team's details below — these aren't in the codebase, so they're left as placeholders.

Name	Degree Programme	Student ID	Role
[Member 1 name]	[e.g. BSc Computer Science]	[ID]	[Role]
[Member 2 name]	[degree programme]	[ID]	[Role]
[Member 3 name]	[degree programme]	[ID]	[Role]
[Member 4 name]	[degree programme]	[ID]	[Role]

02

Background of the Selected Problem

Presentations remain one of the most common ways to communicate ideas in classrooms, lecture halls, boardrooms and conferences, yet the way presenters control their slides has barely changed: they stay tethered to a keyboard and trackpad, or depend on a separate handheld remote clicker. A clicker is one more device that can be forgotten, run flat, or get lost, and physically operating it pulls the presenter's attention and hands away from the audience. It also offers no way to gesture toward specific content the way a laser pointer does. For presenters with limited hand strength, tremors, or other motor-control difficulty, gripping and precisely clicking a small remote button can itself be a barrier to presenting comfortably.

At the same time, webcams are now built into virtually every laptop, and on-device machine-learning models for real-time hand tracking — such as Google's MediaPipe — have become fast and accurate enough to run directly inside a web browser, with no specialised hardware and no server round-trip. This convergence of ubiquitous webcams and browser-capable hand-tracking ML creates an opening to replace the physical clicker with natural hand gestures, recognised live from the camera the presenter already has, with no extra hardware and no video ever leaving their machine.

03

Problem Statement

Presenters currently have no natural, low-cost, hardware-independent, and privacy-preserving way to navigate their slides and point at on-screen content while moving freely and facing their audience. Existing options — physical clickers, mouse/keyboard control, or cloud-based gesture-recognition APIs — each fall short: clickers require carrying and maintaining extra hardware and can exclude users who struggle to operate small buttons reliably; keyboard/mouse control chains the presenter to a desk; and cloud-based gesture APIs send the presenter's live video to a remote server, raising privacy and latency

concerns. There is a need for a system that recognises a small, reliable vocabulary of hand gestures in real time, entirely on-device, and works with presentations built in any tool the presenter already uses.

04

Objectives of the Proposed AI Solution

1. Detect and track a presenter's hand in real time from a live webcam feed using an on-device machine-learning model.
2. Classify a fixed vocabulary of hand poses — fist, open palm, pointer, thumbs-up, thumbs-down — into presentation-control actions: pause, resume, virtual pointer, next slide, previous slide.
3. Provide a hands-free virtual pointer, driven by index-finger position, so presenters can highlight on-slide content without a laser pointer.
4. Minimise false triggers through confidence thresholds, pose-hold debounce timing, action cooldowns, and frame-edge detection, so gestures fire only when clearly and deliberately performed.
5. Support any existing slide-authoring tool — PowerPoint, Google Slides, Keynote — by accepting a plain exported PDF, avoiding vendor lock-in.
6. Keep the entire pipeline client-side, so webcam video is processed and discarded in the browser and is never uploaded, preserving presenter privacy.
7. Generate live captions of the presenter's speech during the talk, using on-device speech recognition, so deaf and hard-of-hearing audience members can follow along.
8. Give the presenter an instant, offline recovery cue — a ready-to-speak line and reminder points pulled from the current slide's own text — for the moment they lose their train of thought.
9. Let presenters browse and search every slide by its own content and jump directly to any one, rather than only stepping forward and backward one slide at a time — important for handling audience Q&A.
10. Show the presenter a lightweight pace indicator, with an optional per-slide time budget, so they can tell whether they are on schedule without checking a separate clock or breaking eye contact with the audience.

05

Target Users

- **Educators** — lecturers, teachers and trainers presenting in classrooms and lecture halls, often needing to move between a screen, a whiteboard, and students.
- **Public speakers** — conference speakers and corporate presenters who want to move freely across a stage while keeping eye contact with the audience.
- **Students** — students delivering class presentations, project defences, or thesis vivas without access to a dedicated clicker.
- **Motor-accessibility users** — presenters with motor-control limitations who find small handheld clickers difficult to grip or click reliably.

- **Deaf and hard-of-hearing audience members** — attendees who benefit from live captions of the presenter's speech, shown directly on the slide as they speak.

06

Proposed AI Techniques

Computer-vision hand landmark detection

MediaPipe HandLandmarker — a pretrained deep-learning model that outputs 21 3-D key points per hand from every video frame (fingertips, joints, wrist).

Geometric feature-based pose classification

A rule-based classifier computes finger-extension and finger-curl ratios from the 21 landmarks, relative to the wrist and finger joints, then scores five candidate poses per frame and selects the highest-confidence pose above a tunable threshold.

Temporal signal processing over the ML output

Pose-hold debouncing (a pose must be held for a short interval before it fires), action cooldown windows, and exponential-moving-average smoothing with a dead-zone for the virtual pointer — turning noisy frame-by-frame classifications into stable, deliberate control events.

Edge / on-device inference

The model runs entirely client-side via WebAssembly, with GPU acceleration where available and automatic fallback to CPU — there is no server-side inference step.

Automatic speech recognition (live captions)

The browser's built-in Web Speech API converts the presenter's live microphone audio into text continuously during the talk, displayed as an on-slide caption. Recognition runs in the browser itself (Chrome/Edge), not on a remote server.

Deterministic, non-AI conveniences — by design

Three supporting features are deliberately built without any ML model, each because the task underneath has a single correct, well-defined answer that a fixed rule already computes exactly: the rescue cue reads the current slide's own text out of the PDF (via pdf.js) and reformats it into a ready-to-speak line with a fixed heuristic, rather than generating a summary with a language model; the slide navigator and search index every slide's text with a plain substring match, rather than a semantic/embedding-based search; and the pace tracker is a straightforward elapsed-time calculation against an optional target length. None of the three would be improved by an AI model — see the justification below.

07

Justification for Selecting the AI Techniques

- **Pretrained hand-tracking model over training from scratch** — MediaPipe HandLandmarker is a proven, actively maintained model that already delivers real-time, accurate 21-point tracking across hand shapes, skin tones and lighting conditions. Training an equivalent model would demand a labelled dataset and compute budget outside the project's scope, so integrating a pretrained model lets effort focus on the presentation-control logic itself.
- **Rule-based classification over a second trained classifier** — the gesture vocabulary is small and fixed (five poses), so a geometric rule-based layer needs no labelled training data, is fully explainable and tunable (the sensitivity setting maps directly onto a confidence threshold), and adds negligible latency compared with training and shipping another neural network.
- **Debounce, cooldown and smoothing over raw per-frame output** — raw per-frame classification is noisy; a presenter's hand passing briefly through a pose while gesturing naturally must not accidentally change slides. These techniques trade a small amount of responsiveness for reliability, which is the right trade-off for a control interface.
- **On-device inference over a cloud gesture API** — running inference in-browser removes network latency, works without a live connection once loaded, avoids server hosting costs, and — critically — never transmits the presenter's webcam video anywhere, directly addressing the privacy concern inherent to any camera-based system.
- **Browser-native Web Speech API over a cloud speech-to-text API** — it requires no API key, no account, and no cost, and keeps captioning consistent with the rest of the system's on-device, no-backend architecture, at the cost of only running in Chromium-based browsers today.
- **A deterministic rescue cue over a generative-AI one** — the one moment a presenter needs the “I'm stuck” button to work is also the worst possible moment for it to depend on a live network call to an AI service. Extracting the slide's own text and reformatting it with a fixed rule is instant and cannot fail from a dropped connection, a slow response, or an API outage — reliability was prioritised over more natural phrasing for a feature that exists specifically for high-pressure moments.
- **Plain substring search over a semantic/embedding-based search** — matching a presenter's own words against a slide's own text is an exact-match problem, not one that benefits from a language model's fuzziness; a substring search returns a correct result instantly, needs no model download, and cannot return an irrelevant match the way a similarity search occasionally can.
- **A simple elapsed-time calculation over any ML-based pacing** — timing a presentation is arithmetic (elapsed time versus a budget), not a pattern-recognition problem, so a model would add complexity and latency without adding accuracy.

08

Expected System Inputs and Outputs

Inputs

Live webcam video stream (browser camera API) · live microphone audio (for captions) · a presentation exported as a PDF · a button press or keyboard shortcut (for the rescue cue) · a typed search query and

thumbnail clicks (for the slide navigator) · an optional target presentation length · user-configurable settings — gesture sensitivity, action cooldown, pointer size, mirror-camera toggle, webcam/landmark overlay visibility, notifications, captions and pace tracker on/off.

Outputs

Classified gesture events (next, previous, pause, resume, pointer start/move/end) · live slide changes in the viewer · an on-screen virtual pointer following the index finger · a hand-landmark skeleton overlay · live speech captions rendered on the slide · an on-slide rescue cue (a suggested line plus reminder points) · a searchable thumbnail grid of the deck · an elapsed-time / pace indicator · status indicators and notifications (camera status, current gesture, paused/active state).

09

Proposed Tools and Technologies

- React 19 with Vite (frontend framework & build tool)
- Tailwind CSS (styling)
- MediaPipe Tasks Vision — HandLandmarker (hand-tracking AI)
- WebAssembly / WebGL (on-device inference runtime, GPU with CPU fallback)
- pdf.js (pdfjs-dist) — in-browser PDF rendering and text extraction
- Web Speech API (SpeechRecognition) — live captioning
- React Router (routing)
- lucide-react (icons)
- oxlint (code quality)

The stack is entirely client-side: the finished build is static and hostable on any static web host — no backend server is required, since both the PDF rendering and hand-tracking inference happen in the browser.

10

Initial System Diagram / Workflow

1. Presenter creates slides in PowerPoint / Google Slides / Keynote
2. Export the deck as a PDF
3. Upload the PDF to AccessPresent AI
4. pdf.js renders the slides in-browser
5. Presenter enables the webcam
6. MediaPipe HandLandmarker tracks 21 hand landmarks per frame (WASM / GPU)
7. Hand-geometry classifier scores finger extension / curl ratios
8. Confidence check: below threshold → loop back to tracking; above threshold → continue
9. Gesture engine applies pose-hold debounce + cooldown

10. A gesture event is emitted: next/previous slide, pause/resume, or pointer move
11. Presentation state updates and the viewer re-renders

The loop runs continuously while presenting: each webcam frame is tracked, classified, and — once it clears the confidence threshold and hold/cooldown checks — turned into a single control action that updates the on-screen presentation state.

Live captions (runs in parallel, independent of the gesture loop)

1. Presenter enables captions (setting or control-bar toggle)
2. Browser microphone stream is opened
3. Web Speech API transcribes speech continuously
4. Latest transcript segment is rendered as an on-slide caption

Rescue cue (on demand)

1. Presenter presses the “I’m stuck” button or the R key
2. pdf.js extracts the current slide's text content
3. Text lines are reordered top-to-bottom, left-to-right and grouped into rows
4. A fixed heuristic turns the first line into a “say this” opener and the next lines into reminder points
5. The cue is cached for that slide and shown instantly on repeat presses

Slide navigator & search (on demand)

1. Presenter opens the slide browser from the control bar
2. Every slide's text is extracted and cached in the background (reusing the rescue cue's cache for any page already visited)
3. Thumbnails render for every slide via pdf.js
4. Presenter types a search query (optional)
5. Matching slides are filtered down with the matching line shown under each thumbnail
6. Presenter clicks a thumbnail; the presentation jumps directly to that slide

Pace tracker (continuous, once presenting starts)

1. Presentation start time and current-slide start time are recorded
2. A one-second tick updates the on-screen elapsed-time display
3. If a target length is set, it is divided by the slide count into a per-slide budget
4. The indicator's colour reflects on-pace / running long / well over, based on that budget
5. The current-slide timer resets whenever the presenter moves to a different slide

Work Division Among Members

Suggested split by system module, mapped onto the codebase's actual structure — replace names once roles are agreed.

Module	Scope	Assigned to
UI & presentation module	Pages, layout, PDF upload & viewer, presentation controls	[Member name]
Gesture recognition engine	MediaPipe integration, hand-geometry classifier, gesture-engine hook	[Member name]
Accessibility & assist features	Live captions (Web Speech API), rescue-cue text extraction and heuristic	[Member name]
Productivity & navigation features	Slide navigator, thumbnail rendering, deck-wide search, pace tracker	[Member name]
State management & settings	Context providers, settings UI, sensitivity/cooldown tuning, integration	[Member name]
Testing, docs & delivery	Usability/accuracy testing, report writing, references, final slide deck	[Member name]

12

Timeline

Twelve-week plan — shift the week numbers onto your course's actual calendar.

Weeks	Milestone
01–02	Requirement analysis, literature review, tool & environment setup
03	System design — architecture, gesture vocabulary, UI wireframes
04–06	Core development — hand-tracking integration, gesture classification, PDF upload/viewer
07	Virtual pointer, pause/resume logic, settings module
08	End-to-end integration of all modules
09	Testing — gesture-accuracy testing, usability testing, bug fixing
10	Refinement — accessibility polish, performance tuning
11	Documentation — final report
12	Presentation preparation, rehearsal, buffer

References

- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. arXiv:2006.10214.
- Lugaresi, C., Tang, J., Nash, H., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.
- Google AI Edge. MediaPipe Hand Landmarker task guide. ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- Rautaray, S. S., & Agrawal, A. (2015). Vision based hand gesture recognition for human–computer interaction: a survey. *Artificial Intelligence Review*, 43(1), 1–54.
- Mitra, S., & Acharya, T. (2007). Gesture Recognition: A Survey. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, 37(3), 311–324.
- Mozilla. PDF.js documentation. mozilla.github.io/pdf.js
- W3C Community Group. Web Speech API specification. wicg.github.io/speech-api
- React documentation. react.dev

Note: Add any course-specific readings your supervisor requires, and format citations in whichever style (APA/IEEE/etc.) your institution mandates.